

Sistemi Operativi – a.a. 2025/2026

prova di laboratorio

– 23 luglio 2026 –

Creare un programma `matrix-type-checker.c` in linguaggio C che accetti invocazioni sulla riga di comando del tipo:

```
matrix-type-checker <file-bin-1> ... <file-bin-N>
```

Il programma prende in input $N \geq 1$ file binari contenenti sequenze di matrici 3×3 di numeri interi senza segno (valori compresi tra 0 e 99) e dovrà classificare ciascuna matrice come "semi-magica", "simmetrica" o "spuria", verificando inoltre che il numero di matrici semi-magiche e simmetriche effettivamente rilevato coincida con quello dichiarato nell'header di ogni file.

Una matrice 3×3 si dice **semi-magica** se tutte le sue righe e tutte le sue colonne hanno la stessa somma (detta "costante semi-magica"); le diagonali non sono considerate. Ad esempio:

30	40	29
35	25	39
34	34	31

10	27	23
32	18	10
18	15	27

sono semi-magiche con costante 99 e 60, rispettivamente.

Una matrice 3×3 si dice **simmetrica** se $a[i][j] = a[j][i]$ per ogni coppia di indici i, j ; in altre parole, gli elementi sono speculari rispetto alla diagonale principale. Ad esempio:

15	42	7
42	88	33
7	33	61

50	11	30
11	25	8
30	8	99

sono simmetriche.

Si assuma che nessuna matrice nei file di input possa essere contemporaneamente semi-magica e simmetrica.

Ogni file binario ha la seguente struttura:

- un header di 8 byte contenente due interi senza segno a 32 bit little-endian (usare il tipo `C uint32_t` in `stdint.h`): il primo dichiara il numero di matrici semi-magiche, il secondo il numero di matrici simmetriche presenti nel file;
- una sequenza di matrici 3×3 da 9 byte ciascuna, memorizzate per righe; ogni elemento della matrice è un byte senza segno (`uint8_t`), con valori compresi tra 0 e 99.

Il numero di matrici contenute in un file si ricava dalla dimensione dello stesso: $(\text{dimensione} - 8) / 9$.

Sono disponibili dei [file di esempio](#): `matrices-A.bin`, `matrices-B-bad.bin`, `matrices-C.bin`.

In particolare al suo avvio il programma creerà $N + 2$ thread ausiliari:

- N thread **lettori** che si occuperanno, rispettivamente, di leggere uno dei file indicati sulla riga di comando tramite la **mappatura dei file in memoria** (`mmap`). Ogni lettore, per ciascuna matrice letta, richiederà una verifica ad entrambi i thread verificatori (prima uno, poi l'altro), depositando la matrice nello slot del verificatore e rimanendo in attesa dell'esito; in base ai risultati, se la matrice risulta di uno dei due tipi, il lettore inserirà un elemento nella coda dei risultati destinata al thread principale;
- 2 thread **verificatori** specializzati (`VERIF-SEMI` per la proprietà semi-magica, `VERIF-SIMM` per quella di simmetria) che si occuperanno, uno alla volta, di verificare la matrice depositata nel proprio slot dal lettore richiedente, comunicandogli l'esito.

Le strutture dati condivise saranno:

- uno **slot** per ciascun verificatore, contenente la matrice 3×3, le informazioni necessarie per identificarla (file di provenienza, numero progressivo) e un campo per l'esito della verifica; lo slot è usato per la comunicazione tra il lettore richiedente e il verificatore in modalità **rendezvous**: il lettore deposita la matrice e attende l'esito, il verificatore preleva, verifica, scrive l'esito nello slot e segnala il lettore; si consiglia l'uso di un campo di stato con i valori EMPTY, TO_VERIFY, VERIFIED per coordinare le transizioni tra lettore e verificatore;
- una **coda dei risultati** con capienza massima di 5 elementi, usata dai lettori per comunicare al thread principale le matrici classificate (non spurie);
- mutex e **variabili condizione**: il loro numero, comunque minimale, e le modalità d'uso sono da determinare da parte dello studente;
- eventuali flag/contatori di fine-lavoro.

Il singolo elemento della coda dei risultati conterrà le seguenti informazioni:

- l'identificativo del file di provenienza;
- il tipo rilevato (semi-magico o simmetrico).

Il thread principale/padre si occuperà di estrarre i risultati inseriti nella coda dai lettori, aggiornando i conteggi dei due tipi per ciascun file; al termine, per ogni file, confronterà il numero di matrici semi-magiche e simmetriche rilevato con quello dichiarato nell'header, riportando l'esito della verifica (superata o fallita) e il riepilogo finale.

L'uso di mutex, variabili condizione e della mappatura dei file in memoria (`mmap`) è obbligatorio: soluzioni che non utilizzino questi strumenti saranno considerate non corrette.

Tutti i thread dovranno terminare spontaneamente alla fine dei lavori e non si dovranno usare strutture dati con visibilità globale. In caso di output interlacciati tra thread, può essere utile il lock dello standard output tramite le funzioni `flockfile(stdout)` e `funlockfile(stdout)`. È necessario rispettare fedelmente la struttura dell'output riportato nell'esempio a seguire. Codici sorgente con errori che bloccano la compilazione e pregiudicano la generazione del codice binario con compilatori standard (gcc o clang) non saranno valutati.

Tempo: 2 ore e 30 minuti

La struttura dell'output atteso è la seguente:

```
$ ./matrix-type-checker matrices-A.bin matrices-B-bad.bin matrices-C.bin
```

```
[MAIN] creazione di 3 thread lettori e 2 thread verificatori
[READER-1] file 'matrices-A.bin': 100 matrici totali, 40 semi-magiche e 35 simmetriche dichiarate
[READER-1] matrice candidata n.1
[READER-2] file 'matrices-B-bad.bin': 80 matrici totali, 30 semi-magiche e 25 simmetriche dichiarate
[VERIF-SEMI] matrice n.1 del file 'matrices-A.bin': (30, 40, 29) (35, 25, 39) (34, 34, 31) -> semi-magica
[READER-2] matrice candidata n.1
[VERIF-SIMM] matrice n.1 del file 'matrices-A.bin': (30, 40, 29) (35, 25, 39) (34, 34, 31) -> no
[VERIF-SEMI] matrice n.1 del file 'matrices-B-bad.bin': (12, 5, 83) (44, 67, 3) (50, 22, 20) -> no
[READER-1] matrice candidata n.2
[VERIF-SIMM] matrice n.1 del file 'matrices-B-bad.bin': (12, 5, 83) (44, 67, 3) (50, 22, 20) -> no
[VERIF-SEMI] matrice n.2 del file 'matrices-A.bin': (15, 42, 7) (42, 88, 33) (7, 33, 61) -> no
[READER-3] file 'matrices-C.bin': 120 matrici totali, 50 semi-magiche e 45 simmetriche dichiarate
[READER-2] matrice candidata n.2
[VERIF-SIMM] matrice n.2 del file 'matrices-A.bin': (15, 42, 7) (42, 88, 33) (7, 33, 61) -> simmetrica
[READER-1] matrice candidata n.3
[VERIF-SEMI] matrice n.3 del file 'matrices-A.bin': (5, 3, 8) (7, 2, 1) (4, 6, 9) -> no
[VERIF-SIMM] matrice n.3 del file 'matrices-A.bin': (5, 3, 8) (7, 2, 1) (4, 6, 9) -> no
[READER-3] matrice candidata n.1
...
[READER-2] terminazione con 80 matrici lette
[READER-1] terminazione con 100 matrici lette
[READER-3] terminazione con 120 matrici lette
[VERIF-SEMI] terminazione con 300 verifiche effettuate
[VERIF-SIMM] terminazione con 300 verifiche effettuate
[MAIN] file 'matrices-A.bin': 40 semi-magiche e 35 simmetriche su 100 matrici -> verifica superata
(dichiarate: 40 semi-magiche, 35 simmetriche)
[MAIN] file 'matrices-B-bad.bin': 28 semi-magiche e 25 simmetriche su 80 matrici -> verifica fallita
(dichiarate: 30 semi-magiche, 25 simmetriche)
[MAIN] file 'matrices-C.bin': 50 semi-magiche e 45 simmetriche su 120 matrici -> verifica superata
(dichiarate: 50 semi-magiche, 45 simmetriche)
[MAIN] verifiche superate: 2/3
[MAIN] terminazione
```